



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 2
по курсу «Алгоритмы компьютерной графики»
«Модельно-видовые и проективные преобразования»

Студент группы ИУ9-42Б Ивенкова О. В.

Преподаватель Цалкович П. А.

Москва 2025

1 Задача

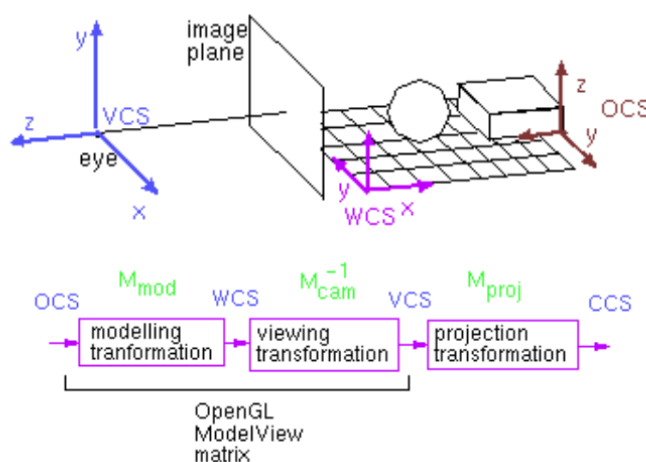
- 1. Определить куб в качестве модели объекта сцены.
- 2. Определить преобразования, позволяющие получить заданный вид проекции (в соответствии с вариантом). Для демонстрации проекции добавить в сцену куб (в стандартной ориентации, не изменяемой при модельно-видовых преобразованиях основного объекта).
- 3. Реализовать изменение ориентации и размеров объекта (навигацию камеры) с помощью модельно-видовых преобразований (без `gluLookAt`). Управление производится интерактивно с помощью клавиатуры и/или мыши.
- 4. Предусмотреть возможность переключения между каркасным и твердотельным отображением модели (`glFrontFace/ glPolygonMode`).

Персональный вариант: двухточечная перспектива.

2 Основная теория

Модельно-видовые преобразования определяют, как объект расположен в мировом пространстве и как он проецируется на экран. В OpenGL они выполняются через матричные операции, применяемые к текущей матрице GL MODELVIEW.

Преобразования в графическом конвейере OpenGL



Функции операций с матрицами преобразований в OpenGL

- Выбор матрицы преобразований для изменения:

```
void glMatrixMode(GLenum mode =
    {GL_MODELVIEW | GL_PROJECTION | GL_TEXTURE})
```

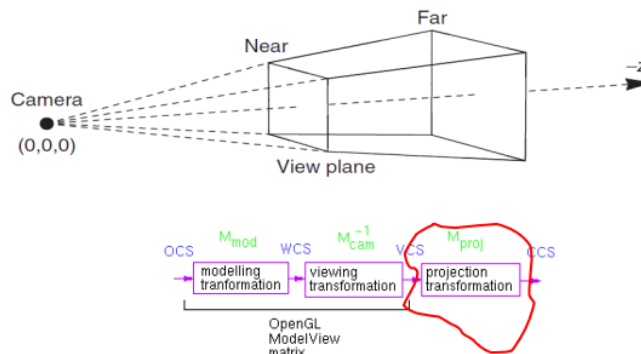
- Основные операции над матрицами:

```
void glLoadIdentity();            $M = I$ 
void glLoadMatrixd(GLdouble m[16]);
void glMultMatrixd(GLdouble m[16]);
```

$$V = C \cdot \begin{bmatrix} m[0] & m[4] & m[8] & m[12] \\ m[1] & m[5] & m[9] & m[13] \\ m[2] & m[6] & m[10] & m[14] \\ m[3] & m[7] & m[11] & m[15] \end{bmatrix} \cdot \begin{bmatrix} v[0] \\ v[1] \\ v[2] \\ v[3] \end{bmatrix}$$

Проективное преобразование и проекция

- проективное преобразование: реализует 3D преобразование, подготавливая модель к переходу к 2D (координата Z сохраняется);
- проекция: переход к двумерной системе координат;
- после проективного преобразования необходимо отбросить координату z и получить значения в оконных координатах;
- объём видимости: ограничивает видимую область пространства;



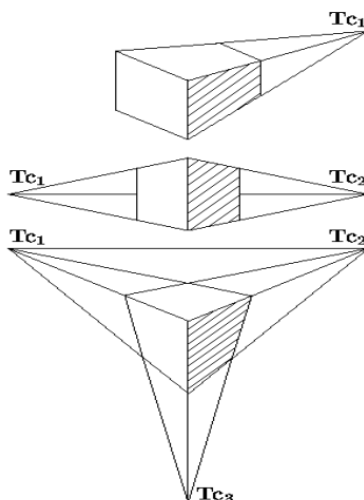
Классификация проекций

- параллельная:
 - ортогональная, ортографическая*;
 - аксонометрическая:
 - прямоугольная *:
 - изометрия;
 - диметрия;
 - триметрия;
 - косоугольная **:
 - кавалье / военная перспектива (горизонтальная изометрия);
 - кабинетная (фронтальная диметрия);
- перспективная (центральная):
 - одноточечная;
 - двухточечная;
 - трехточечная;

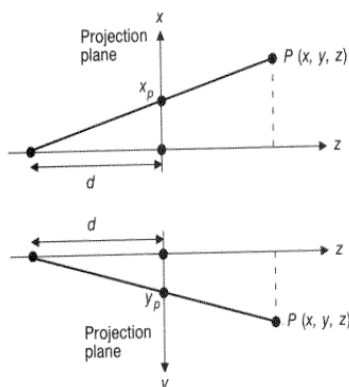
Центральные (перспективные) проекции

Центральные проекции
параллельных прямых, не параллельных плоскости проекции будут сходиться в *точке схода*, количество которых (для прямых, параллельных осям координат), зависит от числа координатных осей, которые пересекает плоскость проекции

$$[T] = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ p & q & r & 1 \end{pmatrix}$$



Перспективная проекция: матричное представление



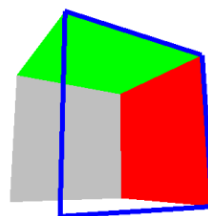
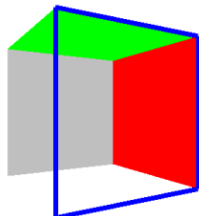
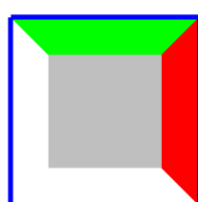
$$M_{per} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1/d & 1 \end{bmatrix}$$

$$x_p = \frac{d \cdot x}{z + d} = \frac{x}{(z/d) + 1}$$

$$y_p = \frac{d \cdot y}{z + d} = \frac{y}{(z/d) + 1}$$

$$\frac{x_p}{d} = \frac{x}{z + d}, \frac{y_p}{d} = \frac{y}{z + d},$$

Перспективная проекция: пример



• одноточечная

$$M_{1p} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -2 & 1 \\ 0 & 0 & -1 & 2 \end{pmatrix}$$

• двухточечная

$$M_{2p} = \begin{pmatrix} 0,87 & 0 & 0,5 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & -1,73 & 1 \\ 0,5 & 0 & -0,87 & 2 \end{pmatrix}$$

• трёхточечная

$$M_{3p} = \begin{pmatrix} 0,87 & 0 & 0,5 & 0 \\ -0,09 & 0,98 & 0,15 & 0 \\ 0,98 & 0,35 & -1,7 & 1 \\ 0,49 & 0,17 & -0,85 & 2 \end{pmatrix}$$

3 Практическая реализация

Исходный код программы представлен в листинге 1.

Листинг 1: Реализация файла main.py

```
1 import glfw
2 from OpenGL.GL import *
3 from OpenGL.GLUT import *
4 from OpenGL.GLU import *
5 import numpy as np
6
7 angle_x, angle_y = 0, 0
8 scale = 0.5
9 wireframe = False
10 vertices = [(-1,-1,-1), (1,-1,-1), (1,1,-1), (-1,1,-1),
11             (-1,-1,1), (1,-1,1), (1,1,1), (-1,1,1)]
12 faces = [(0,1,2,3), (4,5,6,7), (0,1,5,4), (2,3,7,6), (0,3,7,4),
13          (1,2,6,5)]
14
15 projection_matrix = np.array([
16     0.87, 0, 0.5, 0,
17     0, 1, 0, 0,
18     1, 0, -1.73, 1,
19     0.5, 0, -0.87, 2
20 ], dtype=np.float32)
21
22 def draw_static_cube():
23     glPushMatrix()
24     glViewport(0, 0, 400, 600)
25     glMatrixMode(GL_PROJECTION)
26     glLoadIdentity()
27     glLoadMatrixf(projection_matrix.T)
28     glMatrixMode(GL_MODELVIEW)
29     glLoadIdentity()
30     glScalef(0.5, 0.5, 0.5)
31     glColor3f(1, 0, 0.5)
32     glBegin(GL_QUADS)
33     for face in faces:
34         for vertex in face:
35             glVertex3fv(vertices[vertex])
36     glEnd()
37     glPopMatrix()
38
39 def draw_moving_cube():
40     glViewport(400, 0, 400, 600)
41     glMatrixMode(GL_PROJECTION)
```

```

41     glLoadIdentity()
42     glLoadMatrixf(projection_matrix.T)
43     glMatrixMode(GL_MODELVIEW)
44     glLoadIdentity()
45     glRotatef(angle_x, 1, 0, 0)
46     glRotatef(angle_y, 0, 1, 0)
47     glPushMatrix()
48     glScalef(scale, scale, scale)
49     glBegin(GL_QUADS)
50     glColor3f(0, 1, 0.5)
51     for face in faces:
52         for vertex in face:
53             glVertex3fv(vertices[vertex])
54     glEnd()
55     glPopMatrix()
56
57 def display():
58     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
59     draw_static_cube()
60     draw_moving_cube()
61     glfw.swap_buffers(window)
62
63 def key_callback(window, key, scancode, action, mods):
64     global angle_x, angle_y, wireframe, scale
65     if action == glfw.PRESS or action == glfw.REPEAT:
66         if key == glfw.KEY_UP:
67             angle_x -= 5
68         elif key == glfw.KEY_DOWN:
69             angle_x += 5
70         elif key == glfw.KEY_LEFT:
71             angle_y -= 5
72         elif key == glfw.KEY_RIGHT:
73             angle_y += 5
74         elif key == glfw.KEY_EQUAL:
75             scale += 0.1
76         elif key == glfw.KEY_MINUS:
77             scale -= 0.1
78         elif key == glfw.KEY_W:
79             wireframe = not wireframe
80             glPolygonMode(GL_FRONT_AND_BACK, GL_LINE if wireframe else
81                           GL_FILL)
82
83 def scroll_callback(window, xoffset, yoffset):
84     global scale
85     if yoffset > 0:
86         scale += 0.1

```

```

86     else :
87         scale -= 0.1
88
89 def init() :
90     glEnable(GL_DEPTH_TEST)
91
92 if not glfw.init() :
93     raise Exception("GLFW can't be initialized")
94
95 window = glfw.create_window(800, 600, "Lab2", None, None)
96 if not window :
97     glfw.terminate()
98     raise Exception("GLFW window can't be created")
99
100 glfw.make_context_current(window)
101 glfw.set_key_callback(window, key_callback)
102 glfw.set_scroll_callback(window, scroll_callback)
103 init()
104
105 while not glfw.window_should_close(window) :
106     display()
107     glfw.poll_events()
108     #print(f"Scale: {scale}, Angle X: {angle_x}, Angle Y: {angle_y}")
109
110 glfw.terminate()

```

Результат запуска представлен на рисунках 1– 3.

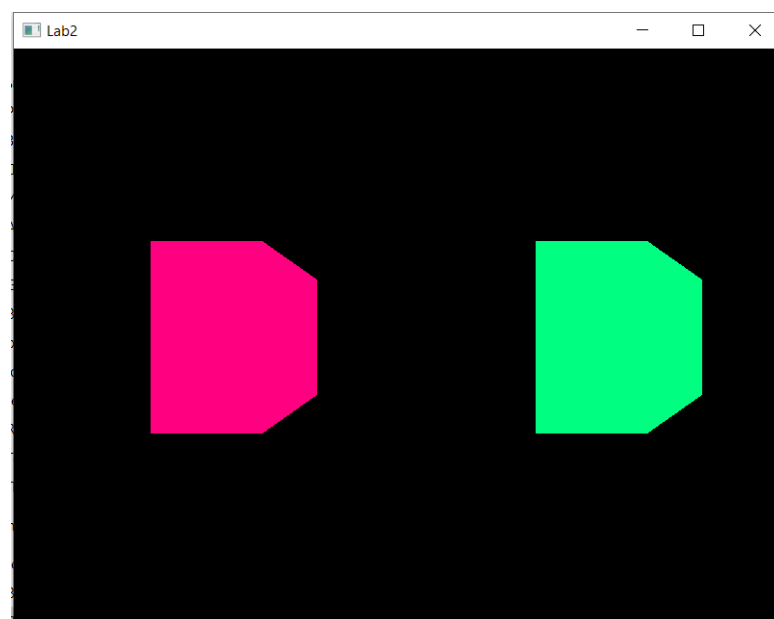


Рис. 1 — Начальное положение обоих кубов

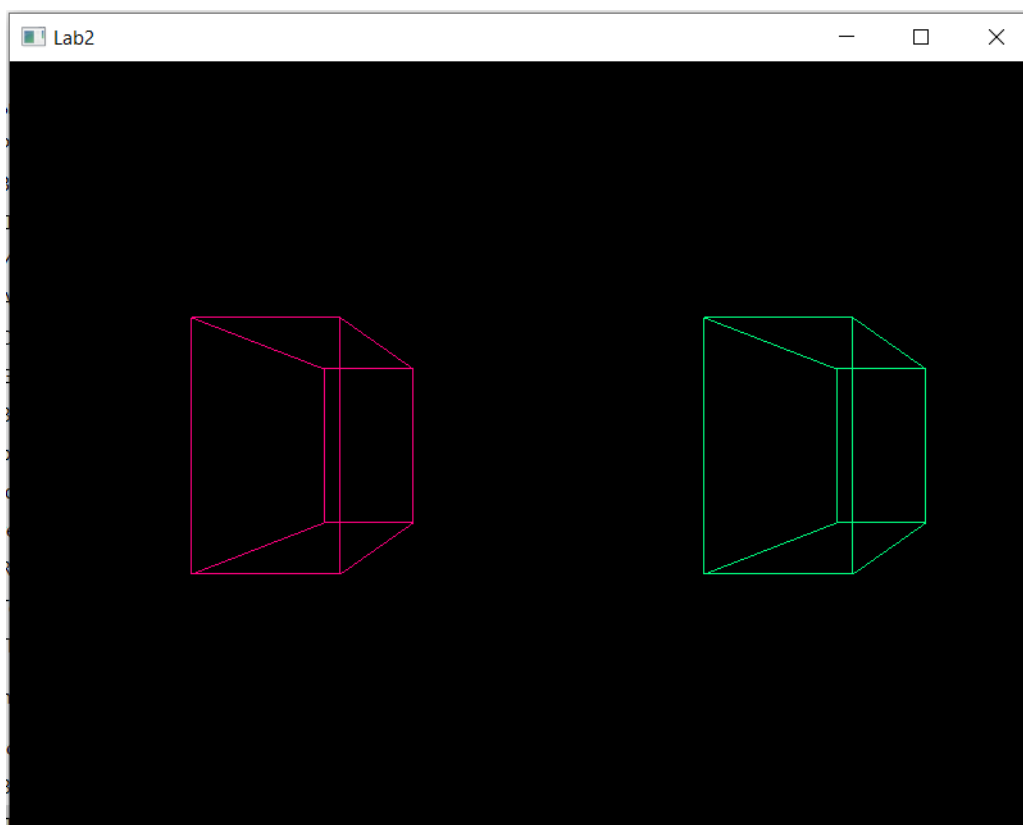


Рис. 2 — Каркасное отображение кубов

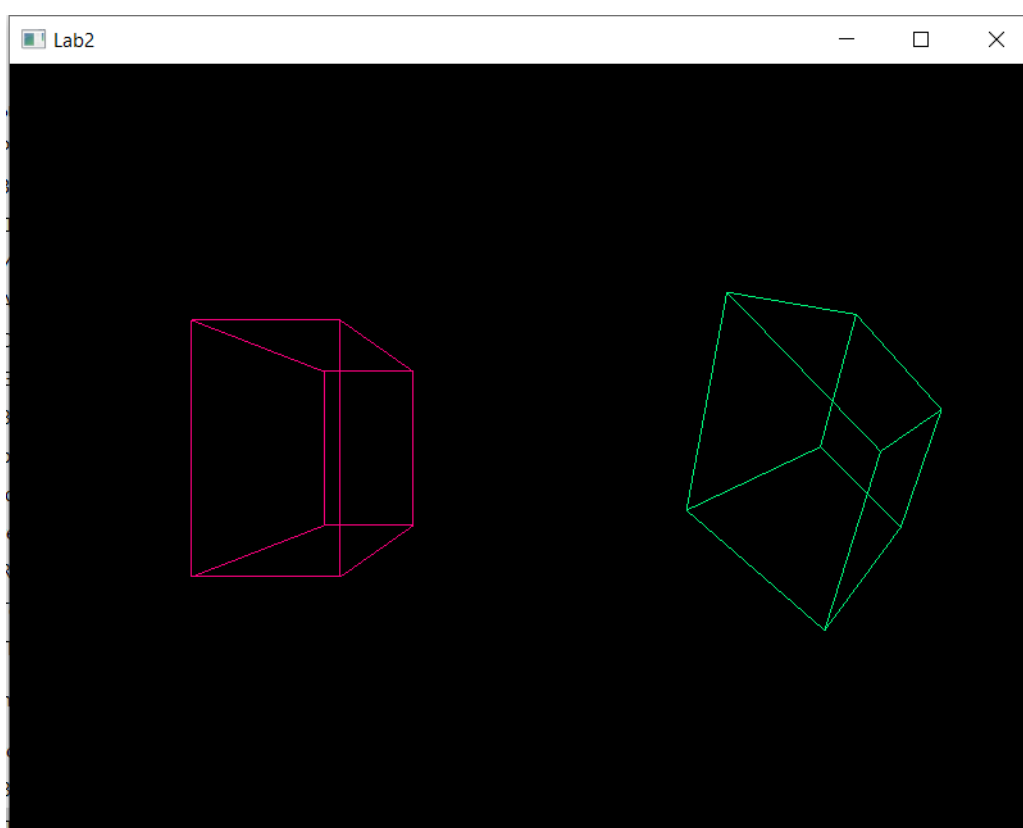


Рис. 3 — Изменение ориентации второго куба

4 Заключение

В ходе выполнения работы был реализован интерактивный рендеринг куба с возможностью управления его ориентацией и масштабом через модельно-видовые преобразования. Проекция объекта задаётся с использованием матрицы двухточечной перспективы, обеспечивающей требуемый вид отображения.

Реализовано переключение между каркасным и твердотельным режимами отображения при помощи `glPolygonMode()`.

Функциональность управления камерой реализована без использования `gluLookAt()`, вместо этого выполняются модельно-видовые преобразования (`glRotatef`, `glScalef`), что соответствует требованиям задания.

В результате разработанный код удовлетворяет поставленным условиям и демонстрирует корректную работу модельно-видовых и проективных преобразований.